IMPLEMENTATION GUIDE · IP–PSTN WEBRTC

Browser Calling with Exotel
from zero to a live call

Account requirements, the complete API surface, exact request and response bodies, and the implementation details required for a working integration. Based on a full production build.

> **Scope.** Turning a web page into a working telephone: obtaining credentials, provisioning an Exotel application, creating a WebRTC agent, registering a browser onto Exotel's SIP network, placing and receiving calls, and recording them. Every API call shown was executed against a live account, and every response body is genuine with secrets removed. The final part collects five implementation details that repay attention early.

DOCUMENT TYPE
Engineering guide and field report

AUDIENCE
Engineers, with plain-language notes throughout

PLATFORM
Exotel IP–PSTN WebRTC, Mumbai region

SDK
`@exotel-npm-dev/exotel-ip-calling-crm-websdk` 1.2.2

VERIFIED
Live account, outbound call completed

STATUS
Production ready

Contents

PART I

Orientation

System overview, the components involved, and the terminology used throughout this document.

1 What browser calling is

A conventional call travels over the telephone network. A WebRTC call travels over the internet. Exotel's IP–PSTN product bridges the two, and the name describes the function: IP on one side, the public switched telephone network on the other.

An agent works in a browser with a headset. When they call a customer's mobile, audio leaves the browser as internet packets, reaches Exotel, and is converted into an ordinary telephone call delivered to the handset. The customer sees your ExoPhone number as the caller ID. In the reverse direction, a customer dials your ExoPhone, Exotel converts that call into internet packets, and the agent's browser rings.

> **IN PLAIN LANGUAGE**
>
> Exotel acts as a translator between two networks. The browser sends audio over the internet; a mobile phone expects audio from the telephone network. Exotel converts between the two in real time, so neither side is aware of the other's format. For the agent this means a computer, a headset and a web page are sufficient. No desk phone, SIM card or telephone line is required.

Two consequences shape the rest of this guide. First, because the browser handles real-time audio, browser rules apply: microphone permission, secure origins and autoplay policy. Second, because Exotel places real telephone calls on your behalf, platform rules apply: account verification, balance, and a provisioned identity for every agent. A substantial part of the integration effort is meeting these two sets of requirements rather than writing application code.

2 System components and the role of the backend

Three components are involved. The distinction that matters is which of them is permitted to hold credentials.

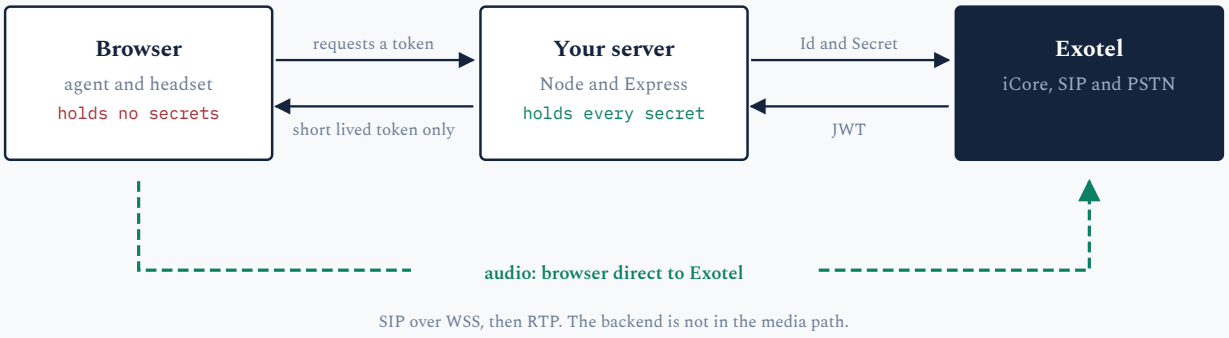

Figure 1. Control and media take separate routes. The backend brokers credentials and never carries voice; audio passes directly between the browser and Exotel. A small server is therefore sufficient, and server latency does not affect call quality.

A backend is required rather than optional. The browser needs an access token to communicate with Exotel, and that token is minted from a secret. A secret placed in JavaScript is published: anyone reading the page source could place calls billed to the account. The secret therefore resides on the server, the server mints tokens, and the browser receives only the token.

3 Vocabulary

TERMS USED THROUGHOUT THIS DOCUMENT

Term	Meaning
PSTN	Public Switched Telephone Network. The telephone system that mobiles and landlines operate on.
WebRTC	The browser technology that carries live audio and video. Built into Chrome, Edge, Safari and Firefox.
SIP	Session Initiation Protocol. The signalling protocol that establishes, answers and ends calls. It does not carry audio.
RTP	The protocol that carries the audio once SIP has established the call.
ExoPhone	An Exotel virtual number. It is the caller ID customers see and the number they dial to reach you.
iCore	Exotel's integrations service. All WebRTC provisioning is performed against it.
Application	An Exotel object representing your integration. It owns agents and settings and has its own credential pair.
User mapping	The record that maps an agent's email address to a SIP identity. Creating one constitutes provisioning an agent.
CallSid	Exotel's unique identifier for a call, required to retrieve status, duration and recordings.

PART II

Exotel account requirements

Account conditions that must be met before integration, and the credentials required for each API.

4 Account prerequisites

Four conditions must hold on the Exotel side before a browser can place a call. Each produces a different error, so checking them in this order will save time.

4.1 The account must not be on trial

A trial account can authenticate and register an application but cannot create a WebRTC agent. The attempt returns:

RESPONSE CREATING AN AGENT ON A TRIAL ACCOUNT

```
{
  "Status": "Failed",
  "Code": 403,
  "Error": "This is a trial account. Operation not permitted."
}
```

This is enabled on Exotel's side. Contact your Exotel representative to have the account enabled for IP–PSTN WebRTC, which normally requires a signed agreement and completed KYC. The account state can be confirmed through the Voice API, where the field to check is `Type`:

GET ACCOUNT STATUS

```
GET https://api.in.exotel.com/v1/Accounts/<sid>.json
Authorization: Basic base64(API_KEY:API_TOKEN)

// response, trimmed
{ "Account": {
    "Type": "Full",            // "Trial" blocks agent creation
    "Status": "active",
    "BillingType": "prepaid",
    "KycStatus": "completed"
} }
```

4.2 The account must have balance

On a prepaid account, calls are placed only while there is balance to bill them against. Everything else continues to operate normally, so credentials, provisioning and SIP registration all report healthy while calls are declined. If calls are declined while registration is green, check the balance first, or ask Exotel support to confirm the account state.

4.3 You need an ExoPhone

The virtual number serves two purposes. On outbound calls it is the caller ID presented to the customer. On inbound calls it is the number customers dial. Each agent carries one on their record as `VirtualNumber`, and agents on the same team commonly share it.

4.4 Inbound routing is configured in the dashboard

Provisioning an agent grants the ability to receive calls but does not determine which calls are delivered. What happens when a customer dials your ExoPhone is decided by a call flow, and that flow is built in the Exotel dashboard rather than through the APIs in this guide.

> **DASHBOARD STEP REQUIRED**
>
> In the Exotel dashboard, attach your ExoPhone to a flow whose applet connects the call to the VoIP or WebRTC agent, identified by the agent's email address. Until this exists, outbound browser calls operate normally and inbound calls are not delivered. No code change is involved.

5 The four credential pairs

Exotel issues several credentials with similar names. Using the wrong one produces an authorisation error that does not identify which. There are four, and they are not interchangeable.

CREDENTIALS AND THEIR PURPOSE

Pair	Used for	Notes
`API Key` `API Token`	Voice API v1	REST over HTTP Basic. Used for account status and call detail records. Not used by WebRTC.
`Client ID` `Client Secret`	Customer entity	Identifies you as the customer. Used once, to register an application. The Client ID is also the CustomerID.
`App ID` `App Secret`	Application entity	Returned when the application is registered. All WebRTC operations use this pair. Store it.
`SIP ID` `SIP Secret`	The browser	Generated by Exotel per agent. Never handled directly; the SDK retrieves it. Never send it to an untrusted client.

5.1 Obtaining each credential

Two of the four pairs are obtained directly. The remaining two are issued by the platform during provisioning and are returned in API responses, so they will not be found in the dashboard.

HOW TO OBTAIN EACH CREDENTIAL

Pair	Source
`API Key` `API Token`	Exotel dashboard, under the API settings for your account. Self-service.
`Client ID` `Client Secret`	**Sent to you by your Exotel representative** as a one page document. See section 5.2.
`App ID` `App Secret`	Returned in the response when the application is registered in section 8.
`SIP ID` `SIP Secret`	Generated automatically when an agent is created in section 10. Retrieved by the SDK.

5.2 Receiving your Client ID and Client Secret

These are issued by Exotel and sent to you as a one page document. Open it and copy both values across.

Store them as `EXOTEL_CLIENT_ID` and `EXOTEL_CLIENT_SECRET` alongside your other Exotel credentials. They are used once, in section 7, to mint the customer token that registers your application. After that point every operation uses the application credentials instead.

> **CREDENTIALS ARE GENERATED ONCE PER ACCOUNT**
>
> They are produced a single time for each Exotel account. Keep the document somewhere durable and treat it as a password: anyone holding the pair can place calls billed to your account.

5.3 Reissues, and anything that is not working

Email `hello@exotel.in` using the template below. Use it for a reissue, and equally if the document has been lost or anything else here is not behaving as described. Generation happens at Exotel's end, so these need a request rather than a retry.

EMAIL TEMPLATE · TO HELLO@EXOTEL.IN

```
To:      hello@exotel.in
Subject: WebRTC credentials | Account SID <your account sid>

Hello Exotel team,

We are integrating the Exotel IP-PSTN WebRTC WebSDK into our application.

Please could you help with the following:
  [ ] Reissue our Client ID and Client Secret
  [ ] We no longer have the document containing them
  [ ] Confirm this account is enabled for IP-PSTN WebRTC

  Account SID      : <your account sid>
  Registered email : <the administrator email on the Exotel account>
  ExoPhone         : <your virtual number>

Thank you,
<your name, company>
```

Put the Account SID in the subject line as well as the body. Nothing can be looked up without it.

> **CONFIRM WHICH VALUE IS THE KEY AND WHICH IS THE TOKEN**
>
> The API Key and API Token are both opaque strings of similar length and the same character set. Nothing in either value identifies which is which, so once they have been copied out of a portal, pasted into a spreadsheet or forwarded by email, the order is easily lost. HTTP Basic auth is order-sensitive: the key is the username and the token is the password.
>
> A reversed pair and an invalid credential produce an identical failure, so the error message does not distinguish between them. Confirm the order with a single read-only request, attempted both ways, before concluding that the credentials are invalid.

DETERMINING THE CORRECT ORDER

```
// GET /v1/Accounts/<sid>.json is read only and has no side effects,
// which makes it a suitable probe. Whichever order returns 200 is correct.

const probe = (user, pass) =>
  fetch(`https://api.in.exotel.com/v1/Accounts/${sid}.json`, {
    headers: { Authorization: "Basic " + btoa(`${user}:${pass}`) }
  }).then(r => r.status);

await probe(A, B);   // 200 means A is the key and B is the token
await probe(B, A);   // 200 means the labels were reversed
// 401 or 403 both ways means the credentials are genuinely invalid
```

6 The two API families

Exotel exposes two API families with separate base URLs and separate authentication schemes. Identifying which family an endpoint belongs to determines how it must be authenticated.

THE TWO FAMILIES

	iCore integrations	Voice API v1
Auth	JWT in `Authorization`	HTTP Basic, key and token
Style	JSON in, JSON out	Form encoded in, JSON out
Casing	`PascalCase` keys	`PascalCase` keys
Owns	Applications, agents, settings, WebRTC calls	Account status, call records
Region	Regional host, `mum1` for Mumbai	`api.in` for India

Everything in Part III is iCore. The Voice API appears twice in this guide: to check the account type in section 4.1 and to read call detail records in section 19.

PART III

Provisioning

Six API calls, in sequence, from initial credentials to an agent capable of registering a browser.

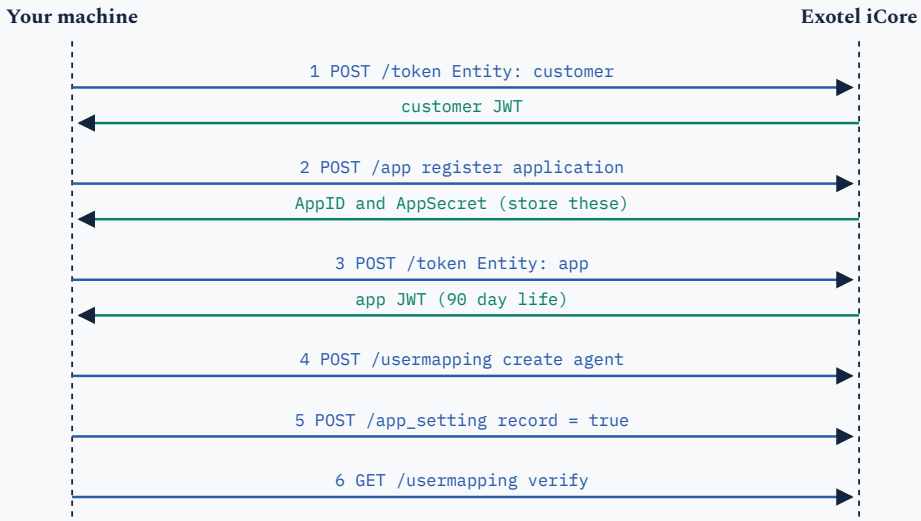

Figure 2. The provisioning sequence. Step 1 uses customer credentials; every step after step 3 uses application credentials. Interchanging them is the most common cause of a 401 at this stage.

7 Step one: mint a customer token

Every iCore call is authorised by a JSON Web Token, and tokens are issued by a single endpoint. The body has exactly three fields and their names matter: `Id`, `Secret` and `Entity`. The `Entity` field selects which kind of token is returned.

POST MINT A CUSTOMER TOKEN

```
POST {icore}/v2/integrations/token
Content-Type: application/json

{
  "Id":     "<your client id>",
  "Secret": "<your client secret>",
  "Entity": "customer"
}
```

200 RESPONSE

```
{
  "RequestId": "ceecabdd-424b-4323-82ea-511697ed9db9",
  "Status":    "Success",
  "Code":      200,
  "Error":     "",
  "Data":      "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJJZCI6..."
}
```

The token is returned in `Data` as a bare string. Every iCore endpoint uses this response shape: a `Status` of `"Success"` or `"Failed"`, a numeric `Code`, an `Error` string that is empty on success, and a `Data` payload. Check `Status` rather than relying on the HTTP code alone.

Pass the token in the `Authorization` header exactly as returned, with no `Bearer` prefix. Decoded, the payload is minimal: `{ "Id": "<app or client id>", "exp": … }`. It carries no account claim, which is relevant to section 21, item five.

8 Step two: register the application

The application owns your agents and settings. It is created once, using the customer token, and your Voice API credentials are supplied at the same time so the application can place calls.

POST REGISTER THE APPLICATION

```
POST {icore}/v2/integrations/app
Authorization: <customer token>
Content-Type: application/json

{
  "AppName":          "webdialer",
  "ExotelAccountSid": "<your account sid>",
  "ExotelApiKey":     "<api key>",
  "ExotelApiToken":   "<api token>",
  "ExotelDomain":     "mumbai",
  "IsActive":         true
}
```

200 RESPONSE

```
{ "Status": "Success", "Code": 200, "Data": {
    "AppID":            "<uuid>",
    "AppSecret":        "<returned once, store it now>",
    "CustomerID":       "<uuid, same as your client id>",
    "AppName":          "webdialer",
    "ExotelAccountSid": "<your account sid>",
    "IsActive":         true
} }
```

> **MAKE THIS STEP IDEMPOTENT**
>
> Registering twice creates a second application, which then competes for the same agents. Before creating, attempt to mint an app token with any stored credentials and call `GET /v2/integrations/app`. If it returns the expected application, skip creation. A provisioning script built this way can be re-run safely at any time.

9 Step three: mint an application token

The same endpoint as step one, with a different entity and different credentials. From this point onward, every WebRTC operation uses this token.

POST MINT AN APPLICATION TOKEN

```
POST {icore}/v2/integrations/token
Content-Type: application/json

{
  "Id":     "<AppID>",
  "Secret": "<AppSecret>",
  "Entity": "app"                    // not "customer"
}
```

The application token observed on a live account was 164 characters with a 90 day expiry. That is long enough that a refresh loop is unnecessary, but a long-lived deployment should mint a fresh token per browser session rather than embedding one.

10 Step four: create the WebRTC agent

This is the step a trial account blocks. It creates a user mapping, which links an email address you choose to a SIP identity Exotel generates. The request body is an array, even for a single agent.

POST CREATE AN AGENT

```
POST {icore}/v2/integrations/usermapping
Authorization: <app token>
Content-Type: application/json

[                                  // an array, not an object
  {
    "AppUserId":        "agent@example.com",
    "AppUsername":      "Agent Name",
    "Email":            "agent@example.com",
    "ExotelAccountSid": "<your account sid>",
    "ExotelUserName":   "Agent Name",
    "VirtualNumber":    "<your exophone>",
    "AgentNumber":      "<optional fallback phone>"
  }
]
```

Reading the agent record back returns the values Exotel generated. This record is the primary diagnostic reference for an agent, so each field is described below.

GET READ THE AGENT BACK

```
GET {icore}/v2/integrations/usermapping?user_id=agent%40example.com
Authorization: <app token>

{ "Status": "Success", "Data": {
    "AppUserId":       "agent@example.com",
    "ExotelUserId":    "<hex id>",
    "VirtualNumber":   "<your exophone>",
    "AgentNumber":     "+91…",          // normalised to E.164
    "SipId":           "sip:<generated>",  // browser SIP username
    "SipSecret":       "<never expose>",
    "SipDeviceID":     "372299",           // the browser device
    "PhoneDeviceID":   "372298",           // created by AgentNumber
    "ActiveDeviceId":  "372299",           // which one rings
    "OutboundActive": true,
    "IsActive":        true
} }
```

VERIFY ACTIVEDEVICEID AFTER SETTING AGENTNUMBER

Supplying `AgentNumber` causes Exotel to create a second device, a phone device, alongside the browser's SIP device. Both exist, but only `ActiveDeviceId` receives calls. Confirm that `ActiveDeviceId` equals `SipDeviceID`, otherwise calls will be delivered to a handset rather than the browser.

11 Step five: turn on recording

Recording is configured at the application level rather than per call. The SDK's call payload is `{customer_id, app_id, to, user_id}`, which has no field for it. Recording is a property of the application and applies to every call that application places.

POST ENABLE RECORDING FOR EVERY CALL

```
GET {icore}/v2/integrations/app_setting       // current value
{ "Data": [ { "Key": "record", "Value": "false" } ] }

POST {icore}/v2/integrations/app_setting
Authorization: <app token>
Content-Type: application/json

{ "Key": "record", "Value": "true" }

// read it back to confirm
{ "Key": "record", "Value": "true" }
```

Values are strings rather than booleans. Send `"true"`, not `true`. Because the setting is application-wide, there is nothing to specify at call time.

12 Step six: verification

Server-side failures are considerably easier to read than browser ones. Confirm all four of the following from a terminal before opening the dialer, because each maps to a specific browser symptom.

PRE-FLIGHT CHECKS AND THE BROWSER SYMPTOM EACH PREVENTS

Check	Expected	If it fails, the browser shows
`POST /token` with Entity app	`Success`	Token request rejected; nothing loads
`GET /app`	`200`	SDK initialise returns nothing, silently
`GET /app_setting`	`200`	SDK initialise aborts before SIP starts
`GET /usermapping`	`Data present`	SDK initialise returns nothing, silently

THE APPLICATION SETTINGS CHECK

The SDK fetches application settings during initialisation and treats a non-OK response as fatal. A freshly registered application that has never had a setting written can return 404 here, and the SDK then aborts with only a console warning, returning `undefined` rather than a phone object. The visible symptom is a dialer that never connects, with no on-screen error. Writing the recording setting in step five ensures this endpoint returns data.

PART IV

Browser integration

SDK initialisation, SIP registration, and the outbound and inbound call flows.

13 What the SDK does on initialisation

Your code calls two methods. Behind them the SDK makes three HTTP requests and then opens a WebSocket. Knowing the order identifies which provisioning step is at fault when initialisation fails.

BROWSER

```
// token and userId are supplied by your own backend
const sdk = new ExotelCRMWebSDK(accessToken, userId, true);

// the third argument requests automatic device registration
const webPhone = await sdk.Initialize(
  handleCallEvents,      // incoming, connected, callEnded, holdtoggle, mutetoggle
  handleRegistration,    // registered, unregistered, sent_request
  handleSession          // ICE state and media permission diagnostics
);

if (!webPhone) { /* provisioning is incomplete; check the console */ }
```

The three requests, in order:

1. `GET /v2/integrations/app` resolves which Exotel account the token belongs to.
2. `GET /v2/integrations/app_setting` loads application settings. A 404 here is fatal.
3. `GET /v2/integrations/usermapping?user_id=<id>` retrieves the agent, including SIP credentials.

The SDK then assembles the SIP account details from these responses. The SIP domain is derived from the account SID, which defines the identity model:

INSIDE THE SDK

```
sipdomain: app.ExotelAccountSid + ".voip.exotel.com"
userName:  userData.SipId.split(":")[1]
secret:    userData.SipSecret
domain:    "voip.in1.exotel.com:443"
security:  "wss"
```

> **INITIALIZE RETURNS UNDEFINED INSTEAD OF THROWING**
>
> If any of the three requests fails, the SDK logs to the browser console and returns `undefined`. It does not throw, so a `try / catch` will not catch it. Always check the return value. This behaviour accounts for most reports of an integration that fails with no visible error.

14 SIP registration over WebSocket

With SIP details in hand, the SDK opens a secure WebSocket and performs a standard SIP registration with digest authentication. The first REGISTER is always rejected; this is the protocol operating correctly rather than an error.

401 THE EXPECTED CHALLENGE

```
SIP/2.0 401 Unauthorized
CSeq: 2 REGISTER
WWW-Authenticate: Digest realm="<account>.voip.exotel.com",
                  nonce="…", qop="auth"
Server: kamailio (5.7.0)
```

200 REGISTERED

```
REGISTER sip:<account>.voip.exotel.com SIP/2.0
Authorization: Digest algorithm=MD5, username="<sip id>",
               realm="<account>.voip.exotel.com", response="…",
               qop=auth, nc=00000001
Contact: <sip:…;transport=wss>;expires=300

SIP/2.0 200 OK
```

The registration callback then fires with `"registered"`. The registration lasts 300 seconds and the SDK refreshes it. Three states are emitted in practice: `sent_request` while in flight, `registered` on success, and `unregistered` on failure or when the transport drops. Treat `unregistered` as temporary and re-register rather than tearing down the session.

15 Placing an outbound call

Exotel places outgoing calls on your behalf rather than dialling directly from the browser. This section describes that flow and the handling it requires.

`webPhone.MakeCall(number, callback)` asks Exotel to place a call. Exotel then brings you into it by sending a SIP INVITE to the browser you registered earlier, and joins that leg to the customer. An outgoing call therefore arrives back at your event handler as an `"incoming"` event.

This design has two practical advantages. Because Exotel originates both legs, caller ID, routing, recording, billing and call detail records are managed centrally, and the browser requires no knowledge of the telephone network. It also means a single event path handles both inbound and outbound calls, so media is connected in one place in your application. The requirement it places on your code is to answer the leg Exotel offers.

POST THE REQUEST MAKECALL ISSUES

```
POST {icore}/v2/integrations/call/outbound_call
Authorization: <app token>
Content-Type: application/json

{
  "customer_id": "<customer id>",
  "app_id":      "<app id>",
  "to":          "<number to dial>",
  "user_id":     "agent@example.com"
}
// no field for caller ID, and none for recording
```

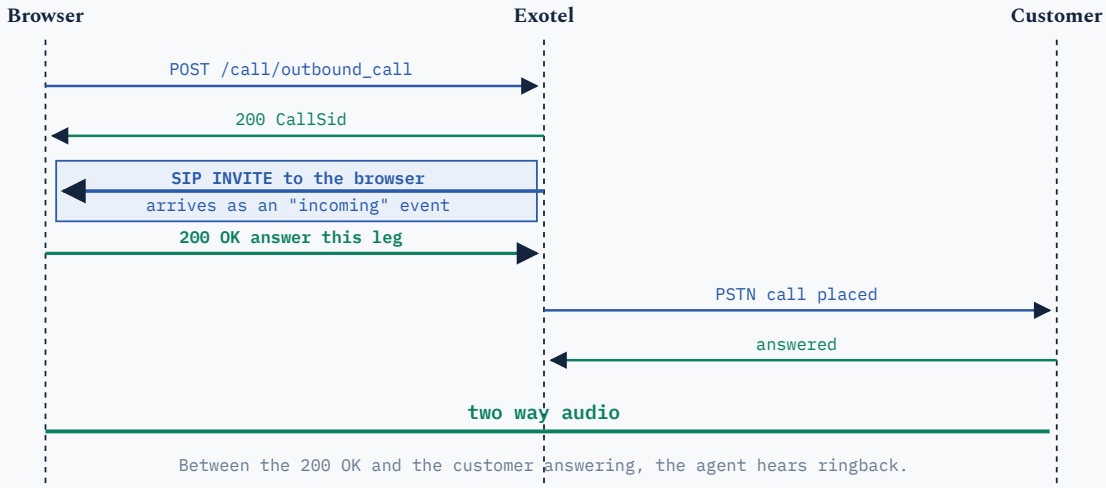

Figure 3. The complete outbound call sequence. The highlighted arrow is the leg that requires handling: Exotel invites the browser into the call the agent requested. Answering it automatically presents the agent with a normal outgoing call.

HANDLING THE RETURN LEG

```
function dial(number) {
  state.pendingOutbound = true;   // set BEFORE calling
  webPhone.MakeCall(number, (status, response) => {
    if (status === "success") log(response.Data.CallSid);
  });
}

function handleCallEvents(event, data) {
  if (event === "incoming") {
    if (state.pendingOutbound) {      // our own call returning
      webPhone.AcceptCall();            // answer without prompting
      return;
    }
    showIncomingPopup(data.callFromNumber);   // a genuine inbound call
  }
}
```

Set the flag before calling `MakeCall` rather than inside its success callback. The INVITE can reach the browser before the HTTP response does, and if the flag is not yet set an incoming call prompt will be shown in error.

> **HANDLING A CANCELLED DIAL**
>
> If the agent hangs up while still dialling there is no SIP session to end, so `HangupCall` has no effect and Exotel's INVITE may still arrive seconds later. Record a short-lived cancellation timestamp and decline any INVITE arriving within that window, otherwise a cancelled call reappears as an unexpected inbound ring.

16 Receiving an inbound call

An inbound call involves no HTTP request. A customer dials the ExoPhone, the dashboard flow routes the call to the VoIP agent, and Exotel sends an INVITE to whichever browser currently holds the registration. The `"incoming"` handler fires with the caller's number in `callFromNumber`. Ring, display a prompt, and call `AcceptCall()` to answer.

The INVITE carries headers the SDK stores, including `X-Exotel-Callsid`, which is the CallSid used to retrieve the call record later.

> **ONE AGENT MEANS ONE REGISTRATION**
>
> An agent is a single SIP identity, and a SIP identity holds one registration. Opening the same agent in two browsers registers the same identity twice, and Exotel rings whichever registered most recently. The resulting failure is difficult to diagnose: calls are delivered to an unattended machine while the intended agent, whose dialer reports a healthy registration, receives nothing. Provision one agent per seat.

17 Microphone access and browser policy

Three browser rules apply, none of which can be overridden from your code.

Secure origin

`getUserMedia` is unavailable outside a secure context, meaning `https://` or `localhost`, which browsers treat as secure by exception. Serving the dialer over plain HTTP on a LAN address provides no microphone at all, and the failure presents as a broken application rather than a policy decision.

Permission requires a user gesture

Browsers present the microphone prompt in response to a user action. Request it deliberately, from a button, rather than allowing the SDK to trigger it during the first call. Query the current state first so the prompt can be skipped when permission is already granted:

REQUESTING MICROPHONE ACCESS

```
const perm = await navigator.permissions.query({ name: "microphone" });

if (perm.state === "granted") { /* open and release, silently */ }
if (perm.state === "prompt")  { /* present your own button first */ }
if (perm.state === "denied")  { /* explain the padlock icon */ }
```

On macOS a second gate applies above the browser: the browser itself must be permitted microphone access in System Settings, under Privacy and Security. If the operating system is blocking it, the browser's own prompt does not appear and the page reports that permission was denied.

Audio playback requires the same gesture

The gesture that grants the microphone also unlocks audio playback. An `AudioContext` created at page load starts suspended, leaving ringtones silent. Create or resume it within the permission click. The SDK plays its own ringtone files on detached audio elements, which are subject to the same policy.

PART V

Operations

Call recording, call detail records, and production deployment requirements.

18 Call recording

The mechanism is covered in section 11. Three operational points are worth stating separately.

First, recording is all or nothing per application. If recording is required on some calls but not others, two applications are needed, and each agent belongs to one application. Second, the setting takes effect immediately for calls placed after it is written and does not apply retroactively. Third, recordings are available against the call in the Exotel dashboard and through the call detail record described next.

> **RECORDING AND CONSENT**
>
> Jurisdictions differ on whether one party or all parties must consent to a recorded call. Because this setting is account-wide and silent, nothing in the product informs an agent that a call is being recorded. If an announcement or consent prompt is required, it belongs in your call flow or agent script, and is worth deciding before enabling recording.

19 Reading call detail records

The Voice API returns the authoritative record of what occurred. It is the fastest way to establish whether a problem lies in the browser or on the network, because it shows what the platform recorded.

GET A CALL RECORD

```
GET https://api.in.exotel.com/v1/Accounts/<sid>/Calls/<CallSid>.json
Authorization: Basic base64(API_KEY:API_TOKEN)

{ "Call": {
    "Sid":           "<CallSid>",
    "From":          "sip:<sip id>",      // the browser leg
    "To":            "<customer number>",
    "PhoneNumberSid": "<exophone>",       // caller ID presented
    "Status":        "completed",
    "Duration":      19,
    "Direction":     "outbound-dial"
} }
```

A `From` value beginning `sip:` confirms that the call originated from a browser rather than a handset, which is a quick way to confirm that the WebRTC path was used.

DO NOT PUT CREDENTIALS IN THE URL

Documentation often shows Voice API calls in the form `https://key:token@api.in.exotel.com/...`. Modern HTTP clients reject this. Node's built-in `fetch` throws outright: *"Request cannot be constructed from a URL that includes credentials."* Use an `Authorization: Basic` header instead. Worth checking early: the request never reaches Exotel, so there is nothing on the platform side to inspect.

20 Production deployment

HTTPS is required

Because `getUserMedia` requires a secure context, HTTPS is required as soon as the dialer is used from a machine other than the one running the server. Any platform that terminates TLS is suitable. When deploying behind a proxy, configure the framework to trust it, otherwise the request is reported as insecure and cookies marked Secure are never set.

Protect the token endpoint

An endpoint that mints application tokens is functionally an endpoint that permits the caller to place calls billed to your account and to read your agents' SIP credentials. On localhost the exposure is limited to the local machine; on a public URL it extends to anyone who can reach it. Place authentication in front of this endpoint before the first deployment.

RECOMMENDED MINIMUM

A shared password, a signed session cookie with an expiry, constant-time comparison, and per-address backoff on failed attempts is sufficient for a small internal tool and takes well under an hour. Leave the status callback route open, since the platform posts to it and it carries no secrets.

Run provisioning locally

Provisioning writes the application credentials back into your environment file. A deployed container's filesystem is discarded on redeploy, so run provisioning from a workstation and copy the resulting values into your platform's environment variables.

PART VI

Implementation notes

Five implementation details, each with the observed behaviour, its cause, and the recommended handling.

21 Five implementation considerations

01 Outgoing calls appear as incoming calls

OBSERVED The agent dials a number and immediately receives an incoming call prompt for it.

CAUSE `MakeCall` asks Exotel to place the call. Exotel then sends a SIP INVITE to the browser, which surfaces as an `"incoming"` event.

HANDLING Track a `pendingOutbound` flag, set before calling `MakeCall`, and answer the matching INVITE automatically. See section 15.

02 Call failures report only the word "Error"

OBSERVED All failures present as the literal string `Error`, with no reason shown in the interface.

CAUSE The SDK reports failures as `new Error(response.statusText)`. The API is served over HTTP/2, where `statusText` is always an empty string. The reason is in the response body, which the SDK discards.

HANDLING Wrap `window.fetch`, and on a failed request to `/call/outbound_call` read the cloned body and extract `error_data.description`.

03 A plus sign in the agent email prevents initialisation

OBSERVED An agent such as `name+webrtc@gmail.com` provisions correctly on the server but the SDK never initialises.

CAUSE The SDK interpolates the user id into a query string without encoding it. In a query string `+` denotes a space, so the lookup is performed for `name webrtc@gmail.com`, returns 404, and initialisation aborts.

HANDLING Pass `encodeURIComponent(userId)` into the SDK constructor. That value is used only to build this URL; the SDK builds its user object from the response. The raw form returns 404; the encoded form returns 200.

04 Setting an agent phone number diverts calls to a handset

OBSERVED After adding `AgentNumber`, calls ring a mobile rather than the browser.

CAUSE Exotel creates a second device, a phone device, and `ActiveDeviceId` determines which one rings.

HANDLING After provisioning, assert `ActiveDeviceId === SipDeviceID`. Leave `AgentNumber` empty for browser-only agents.

05 Account identity is determined by the token

OBSERVED A request configured for one account resolves to a different account name.

CAUSE The SDK derives the account entirely from `GET /v2/integrations/app`, which reflects the token. The `ExotelAccountSid` sent in provisioning bodies is not checked against it, and the SIP realm is built from whatever the token resolved to.

HANDLING After minting a token, call `GET /app`, read the authoritative `ExotelAccountSid`, and reject the request if it is not the expected account. Treat any account SID supplied by a client as untrusted input.

ACCOUNT LEVEL ISSUES

The five items above are handled within your own application. Anything that resides on the account rather than in code, including WebRTC enablement, balance and billing, number provisioning, and how an ExoPhone routes an inbound call, is resolved more quickly with Exotel than inferred from an error message. Contact your Exotel representative with your Account SID and, where relevant, the CallSid.

A useful rule while debugging: if SIP registration is healthy and the failure appears only at call time, examine the account before the code.

22 Error reference

ERRORS OBSERVED ON A LIVE ACCOUNT, AND THEIR MEANING

Code	Message	Meaning and action
`403`	This is a trial account. Operation not permitted.	Agent creation is enabled by Exotel. Contact your Exotel representative for IP–PSTN WebRTC enablement.
`402`	Insufficient balance to make a call. (10720)	Nothing is wrong with the integration. Top up the account, or ask Exotel support to confirm the billing state.
`404`	record not found	User mapping lookup failed. Either the agent is not provisioned, or the id was not URL encoded. See item three.
`404`	No matching results	The Voice API could not find that CallSid, usually a typo. This response also confirms that authentication succeeded.
`401`	SIP/2.0 401 Unauthorized	Not an error. The expected digest challenge during SIP registration, followed by a second REGISTER.
`n/a`	`Initialize()` returned undefined	One of the three initialisation requests failed. Check the console, then repeat the four pre-flight checks in section 12.
`n/a`	Request cannot be constructed from a URL that includes credentials	The HTTP client is rejecting credentials in a URL. Move them to an Authorization header, as in section 19.

23 Verification checklist

Work down the list. Each item depends on those above it, so the first failure is the actual problem and everything below it is consequential.

- ☐ **Client ID and Secret obtained.** From the document sent by Exotel, section 5.2
- ☐ **Account is Full, active, KYC complete.** `GET /v1/Accounts/<sid>.json`
- ☐ **Account has balance.** Prepaid accounts fail at call time only
- ☐ **Customer token mints.** `POST /token` with `Entity: customer`
- ☐ **Application exists.** `GET /app` returns the expected AppID and account SID
- ☐ **App token mints.** `POST /token` with `Entity: app`
- ☐ **Settings endpoint answers.** `GET /app_setting` returns 200, not 404
- ☐ **Recording is enabled.** `record` is the string `"true"`
- ☐ **Agent exists.** `GET /usermapping` returns Data, with the id URL encoded

- [ ] **Browser is the active device.** `ActiveDeviceId === SipDeviceID`
- [ ] **Page is on a secure origin.** HTTPS, or localhost
- [ ] **Microphone granted.** And on macOS, granted to the browser by the OS
- [ ] **SIP registers.** REGISTER, 401, REGISTER, 200 OK
- [ ] **Outgoing call connects.** And the return INVITE is answered automatically
- [ ] **Call record confirms it.** `From` begins with `sip:`
- [ ] **Inbound flow configured.** Dashboard, ExoPhone to VoIP agent
- [ ] **Token endpoint authenticated.** Before the first public deployment

APPENDICES

Reference

Consolidated endpoint reference and glossary.

A Endpoint reference

`{icore}` denotes the regional integrations host. The Voice API base is `https://api.in.exotel.com`. Region strings differ for other Exotel regions.

ICORE INTEGRATIONS

Method	Path	Auth	Purpose
POST	/v2/integrations/token	none	Mint a JWT. The body selects customer or app entity.
POST	/v2/integrations/app	customer	Register an application. Returns AppID and AppSecret.
GET	/v2/integrations/app	app	Resolve which account the token belongs to.
POST	/v2/integrations/usermapping	app	Create agents. The body is an array.
GET	/v2/integrations/usermapping?user_id=	app	Read an agent. URL encode the id.
GET	/v2/integrations/app_setting	app	Read application settings.
POST	/v2/integrations/app_setting	app	Write one setting as a Key and Value pair.
POST	/v2/integrations/call/outbound_call	app	Place a call. Exotel then invites the browser.

VOICE API V1, HTTP BASIC AUTH

Method	Path	Purpose
GET	/v1/Accounts/<sid>.json	Account type, status, billing type, KYC status.
GET	/v1/Accounts/<sid>/Calls/<CallSid>.json	Call detail record: status, duration, direction, price.

B Glossary

API A defined way for two systems to communicate, so one program can ask another to perform an action.

Token A temporary credential. Rather than sending a password with every request, it is exchanged once for a token that expires.

Provisioning Configuring an account so it is ready for use. Here, registering a person as permitted to make calls from a browser.

Agent The person making or receiving calls, and the record representing them in Exotel.

ExoPhone	The business telephone number provided by Exotel. Customers see it and can dial it.
Registration	The browser informing Exotel that it is present and ready to receive calls. If it lapses, calls cannot be delivered.
Inbound and outbound	Calls arriving from customers, and calls placed to them.
Prepaid balance	Funds loaded onto the Exotel account in advance. When exhausted, calls stop.
HTTPS	The secure form of a web address. Browsers do not grant microphone access to pages that are not served over it.

Every request and response body in this document was captured from a live Exotel account during a working integration. Secrets, SIP passwords, application secrets and account identifiers have been replaced with placeholders.